

Лабораторная работа № 4

Линейная алгебра

Доберштейн А. С.

Российский университет дружбы народов, Москва, Россия

Информация

- Доберштейн Алина Сергеевна
- НФИбд-02-22
- Российский университет дружбы народов
- 1132226448@pfur.ru

Цель работы

Основной целью работы является изучение возможностей специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.

1. Используя Jupyter Lab, повторить примеры из раздела 4.2.
2. Выполнить задания для самостоятельной работы.

Выполнение лабораторной работы

Поэлементные операции над многомерными массивами

```
[1]:
a = rand(1:20, (4,3))

[1]:
4x3 Matrix{Int64}:
 15  15   8
 14   2   3
 19   1   4
  7   4  10

[3]:
sum(a), sum(a, dims=1), sum(a, dims=2)

[3]:
(102, [55 22 25], [38; 19; 24; 21;;])

[4]:
prod(a), prod(a, dims=1), prod(a, dims=2)

[4]:
(3217536000, [27930 120 960], [1800; 84; 76; 280;;])
```

Рис. 1: Поэлементные операции над многомерными массивами

Поэлементные операции над многомерными массивами

```
[5]:  
import Pkg  
Pkg.add("Statistics")  
using Statistics  
  
Updating registry at `C:\Users\MI\.julia\registries\General.toml`  
Resolving package versions...  
Updating `C:\Users\MI\.julia\environments\v1.11\Project.toml`  
[10745b16] + Statistics v1.11.1  
No Changes to `C:\Users\MI\.julia\environments\v1.11\Manifest.toml`  
  
[6]:  
mean(a), mean(a, dims=1), mean(a, dims=2)  
  
[6]:  
(8.5, [13.75 5.5 6.25], [12.666666666666666; 6.333333333333333; 8.0; 7.0;;])
```

Рис. 2: Поэлементные операции над многомерными массивами

Транспонирование, след, ранг, определитель и инверсия матрицы

[7]:

```
import Pkg
Pkg.add("LinearAlgebra")
using LinearAlgebra
```

```
Resolving package versions...
Updating `C:\Users\MI\.julia\environments\v1.11\Project.toml`
[37e2e46d] + LinearAlgebra v1.11.0
No Changes to `C:\Users\MI\.julia\environments\v1.11\Manifest.toml`
```

[8]:

```
b = rand(1:20, (4,4))
```

[8]:

```
4x4 Matrix{Int64}:
 6 15 16 12
13  5  5  4
 6  2  1 11
 3  4 17 13
```

[9]:

```
transpose(b)
```

[9]:

```
4x4 transpose{::Matrix{Int64}} with eltype Int64:
 6 13  6  3
15  5  2  4
16  5  1 17
12  4 11 13
```

Рис. 3: Транспонирование, след, ранг, определитель и инверсия матрицы

Транспонирование, след, ранг, определитель и инверсия матрицы

[10]:

```
tr(b)
```

[10]:

25

[11]:

```
diag(b)
```

[11]:

4-element Vector{Int64}:

6

5

1

13

[12]:

```
rank(b)
```

[12]:

4

[13]:

```
inv(b)
```

[13]:

4x4 Matrix{Float64}:

-0.0308512 0.0917122 -0.00229047 0.00219698

0.0977423 -0.0269247 0.00271117 -0.0842332

-0.01809 0.026504 -0.0755855 0.0725004

0.000701164 -0.0475389 0.0985369 0.00752583

Транспонирование, след, ранг, определитель и инверсия матрицы

```
[14]:  
det(b)  
  
[14]:  
21393.0  
  
[15]:  
pinv(a)  
  
[15]:  
3x4 Matrix{Float64}:  
 0.000117046  0.0287047  0.0405749 -0.024935  
 0.0838065   -0.0145833 -0.0379161 -0.0475038  
 -0.0332442  -0.0177878 -0.0108536  0.136273
```

Рис. 5: Транспонирование, след, ранг, определитель и инверсия матрицы

Вычисление нормы векторов и матриц, повороты, вращения

Евклидова норма:

$$\|\vec{X}\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2};$$

p-норма:

$$\|\vec{A}\|_p = \left(\sum_{i=1}^n |a_i|^p \right)^{1/p}.$$

[20]:

```
X = [2, 4, -5]
Y = [1, -1, 3]
```

[20]:

```
3-element Vector{Int64}:
 1
-1
 3
```

[18]:

```
norm(X)
```

[18]:

```
6.708203932499369
```

[19]:

```
p = 1
norm(X, p)
```

[19]:

```
11.0
```

Вычисление нормы векторов и матриц, повороты, вращения

[21]:

```
norm(X-Y)
```

[21]:

9.486832980505138

[22]:

```
sqrt(sum((X-Y).^2))
```

[22]:

9.486832980505138

[23]:

```
acos((transpose(X)*Y)/(norm(X)*norm(Y)))
```

[23]:

2.4404307889469252

[24]:

```
d = [5 -4 2; -1 2 3; -2 1 0]
```

[24]:

```
3x3 Matrix{Int64}:  
 5  -4  2  
-1   2  3  
-2   1  0
```

[25]:

```
opnorm(d)
```

[25]:

7.147682841795258

Вычисление нормы векторов и матриц, повороты, вращения

[26]:

```
opnorm(d, p)
```

[26]:

8.0

[27]:

```
rot180(d)
```

[27]:

```
3x3 Matrix{Int64}:  
 0  1 -2  
 3  2 -1  
 2 -4  5
```

[29]:

```
reverse(d, dims=1)
```

[29]:

```
3x3 Matrix{Int64}:  
-2  1  0  
-1  2  3  
 5 -4  2
```

[30]:

```
reverse(d, dims=2)
```

[30]:

```
3x3 Matrix{Int64}:  
 2 -4  5  
 3  2 -1  
 0  1 -2
```

Матричное умножение, единичная матрица, скалярное произведение

[33]:

```
A = rand(1:10, (2,3))  
B = rand(1:10, (3,4))
```

[33]:

```
3x4 Matrix{Int64}:  
 1  8 10  7  
 9  6  6  6  
 5  2  8  3
```

[34]:

```
A*B
```

[34]:

```
2x4 Matrix{Int64}:  
123 114 180 117  
 74  62 110  66
```

[35]:

```
Matrix{Int}(I, 3, 3)
```

[35]:

```
3x3 Matrix{Int64}:  
 1  0  0  
 0  1  0  
 0  0  1
```

[178]:

```
dot(X, Y), X'Y
```

[178]:

```
(-17, -17)
```

Факторизация. Специальные матричные структуры

```
A = rand(3, 3)
```

```
[38]:
```

```
3x3 Matrix{Float64}:  
 0.130361  0.625315  0.826314  
 0.640786  0.812681  0.00499266  
 0.25978   0.414297  0.28398
```

```
[39]:
```

```
x = fill(1.0, 3)
```

```
[39]:
```

```
3-element Vector{Float64}:  
 1.0  
 1.0  
 1.0
```

```
[40]:
```

```
b = A*x
```

```
[40]:
```

```
3-element Vector{Float64}:  
 1.581989128409948  
 1.4584589248643023  
 0.9580563578159239
```

```
[41]:
```

```
A\b
```

```
[41]:
```

```
3-element Vector{Float64}:  
 1.0000000000000007  
 0.9999999999999994  
 1.0000000000000001
```


Факторизация. Специальные матричные структуры

```
Alu = lu(A)
```

```
[42]:
```

```
LU{Float64, Matrix{Float64}, Vector{Int64}}
```

```
L factor:
```

```
3×3 Matrix{Float64}:
```

```
1.0      0.0      0.0  
0.203439 1.0      0.0  
0.405408 0.184418 1.0
```

```
U factor:
```

```
3×3 Matrix{Float64}:
```

```
0.640786 0.812681 0.00499266  
0.0      0.459984 0.825298  
0.0      0.0      0.129756
```

```
[43]:
```

```
Alu.P
```

```
[43]:
```

```
3×3 Matrix{Float64}:
```

```
0.0 1.0 0.0  
1.0 0.0 0.0  
0.0 0.0 1.0
```

```
[44]:
```

```
Alu.p
```

```
[44]:
```

```
3-element Vector{Int64}:
```

```
2  
1  
3
```

Факторизация. Специальные матричные структуры

Alu.L

[45]:

```
3x3 Matrix{Float64}:  
 1.0      0.0      0.0  
 0.203439  1.0      0.0  
 0.405408  0.184418  1.0
```

[46]:

Alu.U

[46]:

```
3x3 Matrix{Float64}:  
 0.640786  0.812681  0.00499266  
 0.0       0.459984  0.825298  
 0.0       0.0       0.129756
```

[47]:

$A \setminus b$, $Alu \setminus b$

[47]:

```
([1.0000000000000007, 0.9999999999999994, 1.0000000000000004], [1.0000000000000007, 0.9999999999999994, 1.0000000000000004])
```

[48]:

$\det(A)$, $\det(Alu)$

[48]:

```
(-0.038245665935993024, -0.038245665935993024)
```

Факторизация. Специальные матричные структуры

```
Aqr = qr(A)
```

```
[49]:
```

```
LinearAlgebra.QRCompactWY{Float64, Matrix{Float64}, Matrix{Float64}}  
Q factor: 3x3 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}  
R factor:  
3x3 Matrix{Float64}:  
-0.703623 -1.00892 -0.262485  
 0.0      0.452989  0.824723  
 0.0      0.0      0.119992
```

```
[50]:
```

```
Aqr.Q
```

```
[50]:
```

```
3x3 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}
```

```
[51]:
```

```
Aqr.R
```

```
[51]:
```

```
3x3 Matrix{Float64}:  
-0.703623 -1.00892 -0.262485  
 0.0      0.452989  0.824723  
 0.0      0.0      0.119992
```

Рис. 13: Факторизация. Специальные матричные структуры

Факторизация. Специальные матричные структуры

```
Asym = A + A'
```

```
[52]:
```

```
3x3 Matrix{Float64}:  
 0.260722  1.2661   1.08609  
 1.2661   1.62536  0.419289  
 1.08609  0.419289  0.56796
```

```
[53]:
```

```
AsymEig = eigen(Asym)
```

```
[53]:
```

```
Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}}  
values:  
3-element Vector{Float64}:  
 -0.9146620766865166  
  0.5801592806250788  
  2.7885455643426953  
vectors:  
3x3 Matrix{Float64}:  
 -0.805321  0.238993 -0.542531  
  0.318926 -0.596783 -0.736299  
  0.499744  0.765984 -0.404381
```

```
[54]:
```

```
AsymEig.values
```

```
[54]:
```

```
3-element Vector{Float64}:  
 -0.9146620766865166  
  0.5801592806250788  
  2.7885455643426953
```

Факторизация. Специальные матричные структуры

```
AsymEig.vectors

[55]:
3x3 Matrix{Float64}:
-0.805321  0.238993 -0.542531
 0.318926 -0.596783 -0.736299
 0.499744  0.765984 -0.404381

[56]:
inv(AsymEig)*Asym

[56]:
3x3 Matrix{Float64}:
 1.0      1.38778e-15 -3.38618e-15
-2.22045e-15 1.0      3.33067e-15
 1.9984e-15  1.44329e-15 1.0
```

Рис. 15: Факторизация. Специальные матричные структуры

Факторизация. Специальные матричные структуры

```
n = 1000
A = randn(n,n);
```

[58]:

```
Asym = A + A'
```

[58]:

```
1000x1000 Matrix{Float64}:
-2.19479  -1.19271  -0.533717  ...   0.77632  -1.05175  -1.27078
-1.19271   3.48681  -0.95794  ...  -2.32053  -2.43575  -1.84917
-0.533717  -0.95794  -1.92089   ...   2.03246   0.970973   3.0365
-0.793629   0.746374  -0.927323   ...   0.299869  -2.12633   1.58702
 0.508235   0.72203   0.300786   ...  -2.34076   0.501312   0.101021
 0.0353377   0.223765   0.302752   ...   0.284698   0.413091  -0.409259
-0.244464   0.580059  -0.435292   ...   0.903029  -2.47995   0.947742
 0.297977  -0.9202    -1.27663   ...   0.693768  -2.57974  -0.218114
 3.41421    0.204073  -4.06412   ...  -0.249958   0.998755  -0.413718
 2.06395   -0.209414  -0.935789   ...  -2.38471  -0.0473733  -1.24324
 1.83016   -0.626521  -0.00932724 ...   1.63515  -0.877823  -0.0211463
 0.385366  -0.622457   0.648369   ...  -0.666657  -0.143805  -0.0397126
-0.0208005  -0.781617  -0.173238   ...   0.300168  -1.73075  -0.437256
⋮
-0.443598   1.78351   1.93628   ...   2.97376   0.211587   0.604699
 0.610629   1.30669  -1.80773   ...  -1.50994   0.0665224  -1.75304
-0.552511   0.685797  1.00592   ...  -0.577318  -0.638442   2.04725
 0.359329  -1.92356   0.171869   ...   1.11736   3.73648  -1.51708
-0.262325   0.926016  -0.629787   ...   1.51776  -0.540889   0.937301
 0.588058   0.427177  -0.457921   ...   2.16399  -0.0974671   0.359109
 1.36986   -1.89396   1.02321   ...  -1.17409   0.253946   1.90088
-0.519389   1.81656   2.55955   ...  -0.210387  -1.86612   1.1912
 1.79518   -0.268515  0.35924   ...   0.554849  -1.9873   -0.46084
 0.77632  -2.32053   2.03246   ...   1.63586   0.576291  -0.696355
-1.05175  -2.43575   0.970973   ...   0.576291  -0.457038   0.664735
-1.27078  -1.84917   3.0365    ...  -0.696355   0.664735   1.3194
```

Факторизация. Специальные матричные структуры

```
[60]:  
issymmetric(Asym)  
  
[60]:  
true  
  
[61]:  
Asym_noisy = copy(Asym)  
Asym_noisy[1,2] += 5eps()  
  
[61]:  
-1.1927086738197057  
  
[62]:  
issymmetric(Asym_noisy)  
  
[62]:  
false
```

Рис. 17: Факторизация. Специальные матричные структуры

Факторизация. Специальные матричные структуры

```
Asym_explicit = Symmetric(Asym_noisy)
```

```
[63]:
```

```
1000x1000 Symmetric{Float64, Matrix{Float64}}:
```

```
-2.19479 -1.19271 -0.533717 ... 0.77632 -1.05175 -1.27078
-1.19271 3.48681 -0.95794 -2.32053 -2.43575 -1.84917
-0.533717 -0.95794 -1.92089 2.03246 0.970973 3.0365
-0.793629 0.746374 -0.927323 0.299869 -2.12633 1.58702
0.508235 0.72203 0.300786 -2.34076 0.501312 0.101021
0.0353377 0.223765 0.302752 ... 0.284698 0.413091 -0.409259
-0.244464 0.580059 -0.435292 0.903029 -2.47995 0.947742
0.297977 -0.9202 -1.27663 0.693768 -2.57974 -0.218114
3.41421 0.204073 -4.06412 -0.249958 0.998755 -0.413718
2.06395 -0.209414 -0.935789 -2.38471 -0.0473733 -1.24324
1.83016 -0.626521 -0.00932724 ... 1.63515 -0.877823 -0.0211463
0.385366 -0.622457 0.648369 -0.666657 -0.143805 -0.0397126
-0.0208005 -0.781617 -0.173238 0.300168 -1.73075 -0.437256
⋮
-0.443598 1.78351 1.93628 2.97376 0.211587 0.604699
0.610629 1.30669 -1.80773 -1.50994 0.0665224 -1.75304
-0.552511 0.685797 1.00592 ... -0.577318 -0.638442 2.04725
0.359329 -1.92356 0.171869 1.11736 3.73648 -1.51708
-0.262325 0.926016 -0.629787 1.51776 -0.540889 0.937301
0.588058 0.427177 -0.457921 2.16399 -0.0974671 0.359109
1.36986 -1.89396 1.02321 -1.17409 0.253946 1.90088
-0.519389 1.81656 2.55955 ... -0.210387 -1.86612 1.1912
1.79518 -0.268515 0.35924 0.554849 -1.9873 -0.46084
0.77632 -2.32053 2.03246 1.63586 0.576291 -0.696355
-1.05175 -2.43575 0.970973 0.576291 -0.457038 0.664735
-1.27078 -1.84917 3.0365 -0.696355 0.664735 1.3194
```

Рис. 18: Факторизация. Специальные матричные структуры

Факторизация. Специальные матричные структуры

```
import Pkg
Pkg.add("BenchmarkTools")
using BenchmarkTools

Resolving package versions...
Installed BenchmarkTools - v1.6.3
Updating `C:\Users\MI\.julia\environments\v1.11\Project.toml`
[6e4b80f9] + BenchmarkTools v1.6.3
Updating `C:\Users\MI\.julia\environments\v1.11\Manifest.toml`
[6e4b80f9] + BenchmarkTools v1.6.3
[9abbd945] + Profile v1.11.0
Precompiling project...
1718.3 ms ✓ BenchmarkTools
1 dependency successfully precompiled in 4 seconds. 516 already precompiled.

[65]:
@btime eigvals(Asym);

63.067 ms (21 allocations: 7.99 MiB)

[66]:
@btime eigvals(Asym_noisy);

511.761 ms (27 allocations: 7.93 MiB)

[67]:
@btime eigvals(Asym_explicit);

65.078 ms (21 allocations: 7.99 MiB)
```

Рис. 19: Факторизация. Специальные матричные структуры

Факторизация. Специальные матричные структуры

```
n = 1000000;  
A = SymTridiagonal(randn(n), randn(n-1))
```

[68]:

[illegible]

[69]:

```
@btime eigmax(A);
```

402.133 ms (44 allocations: 183.11 MiB)

```
Arational = Matrix{Rational{BigInt}}(rand(1:10, 3, 3))/10
```

[70]:

```
3×3 Matrix{Rational{BigInt}}:  
 1//10  7//10  7//10  
 3//5   1//5   9//10  
 3//10  7//10   1
```

[71]:

```
x = fill(1, 3)
```

[71]:

```
3-element Vector{Int64}:  
 1  
 1  
 1
```

[72]:

```
b = Arational*x
```

[72]:

```
3-element Vector{Rational{BigInt}}:  
 3//2  
 17//10  
 2
```

Рис. 21: Общая линейная алгебра

```
Arational\b
```

```
[73]:
```

```
3-element Vector{Rational{BigInt}}:
```

```
1  
1  
1
```

```
[74]:
```

```
lu(Arational)
```

```
[74]:
```

```
LU{Rational{BigInt}, Matrix{Rational{BigInt}}, Vector{Int64}}
```

```
L factor:
```

```
3×3 Matrix{Rational{BigInt}}:
```

```
1      0      0  
1//6    1      0  
1//2  9//10    1
```

```
U factor:
```

```
3×3 Matrix{Rational{BigInt}}:
```

```
3//5  1//5  9//10  
0     2//3  11//20  
0     0     11//200
```

Рис. 22: Общая линейная алгебра

№1 Произведение векторов

1. Задайте вектор v . Умножьте вектор v скалярно сам на себя и сохраните результат в `dot_v`.

[75]:

```
v = [1, -2, 3, -4, 5]
```

[75]:

```
5-element Vector{Int64}:  
 1  
-2  
 3  
-4  
 5
```

[76]:

```
dot_v = v*v
```

[76]:

```
55
```

[77]:

```
dot_v = dot(v, v)
```

[77]:

```
55
```

Рис. 23: Задание 1.1

№1 Произведение векторов

2. Умножьте v матрично на себя (внешнее произведение), присвоив результат переменной `outer_v`.

[78]:

```
outer_v = v*v'
```

[78]:

```
5x5 Matrix{Int64}:  
 1  -2   3  -4   5  
-2   4  -6   8  -10  
 3  -6   9  -12  15  
-4   8  -12  16  -20  
 5  -10  15  -20  25
```

Рис. 24: Задание 1.2

[88]:

```
function SLAU(A, b)
    m,n = size(A)
    if m == n
        if (det(A) != 0)
            x = A\b
        else
            x = "Нет решений"
        end
    else
        x = A\b
    end
    return x
end
```

[88]:

SLAU (generic function with 1 method)

Рис. 25: Задание 2.1

№2 Системы линейных уравнений

$$\text{a) } \begin{cases} x + y = 2, \\ x - y = 3. \end{cases}$$

[79]:

```
A_a = [1 1; 1 -1]
b_a = [2, 3]
x_a = A_a\b_a
```

[79]:

```
2-element Vector{Float64}:
 2.5
-0.5
```

[89]:

```
SLAU(A_a, b_a)
```

[89]:

```
2-element Vector{Float64}:
 2.5
-0.5
```

[81]:

```
Alu_a = lu(A_a)
xlu_a = Alu_a\b_a
```

[81]:

```
2-element Vector{Float64}:
 2.5
-0.5
```


$$\text{b) } \begin{cases} x + y = 2, \\ 2x + 2y = 4. \end{cases}$$

[90]:

```
A_b = [1 1; 2 2]  
b_b = [2, 4]
```

```
SLAU(A_b, b_b)
```

[90]:

"Нет решений"

$$\text{c) } \begin{cases} x + y = 2, \\ 2x + 2y = 5. \end{cases}$$

[91]:

```
A_c = [1 1; 2 2]  
b_c = [2, 5]
```

```
SLAU(A_c, b_c)
```

[91]:

"Нет решений"

Рис. 27: Задание 2.1

$$\text{d) } \begin{cases} x + y = 1, \\ 2x + 2y = 2, \\ 3x + 3y = 3. \end{cases}$$

[92]:

```
A_d = [1 1; 2 2; 3 3]
```

```
b_d = [1, 2, 3]
```

```
SLAU(A_d, b_d)
```

[92]:

```
2-element Vector{Float64}:
```

```
0.49999999999999999
```

```
0.5
```

$$\text{e) } \begin{cases} x + y = 2, \\ 2x + y = 1, \\ x - y = 3. \end{cases}$$

[93]:

```
A_e = [1 1; 2 1; 1 -1]
```

```
b_e = [2, 1, 3]
```

```
SLAU(A_e, b_e)
```

[93]:

```
2-element Vector{Float64}:
```

```
1.5000000000000004
```

```
-0.9999999999999997
```

$$\text{f) } \begin{cases} x + y = 2, \\ 2x + y = 1, \\ 3x + 2y = 3. \end{cases}$$

[94]:

```
A_f = [1 1; 2 1; 3 2]
```

```
b_f = [2, 1, 3]
```

```
SLAU(A_f, b_f)
```

[94]:

```
2-element Vector{Float64}:
```

```
-0.9999999999999989
```

```
2.9999999999999982
```

Рис. 29: Задание 2.1

$$\text{a) } \begin{cases} x + y + z = 2, \\ x - y - 2z = 3. \end{cases}$$

[96]:

```
A_a = [1 1 1; 1 -1 -2]
b_a = [2, 3]

SLAU(A_a, b_a)
```

[96]:

```
3-element Vector{Float64}:
 2.2142857142857144
 0.35714285714285704
-0.5714285714285712
```

$$\text{b) } \begin{cases} x + y + z = 2, \\ 2x + 2y - 3z = 4, \\ 3x + y + z = 1. \end{cases}$$

[97]:

```
A_b = [1 1 1; 2 2 -3; 3 1 1]
b_b = [2, 4, 1]

SLAU(A_b, b_b)
```

[97]:

```
3-element Vector{Float64}:
-0.5
 2.5
 0.0
```

$$\text{c) } \begin{cases} x + y + z = 1, \\ x + y + 2z = 0, \\ 2x + 2y + 3z = 1. \end{cases}$$

[98]:

```
A_c = [1 1 1; 1 1 2; 2 2 3]
```

```
b_c = [1, 0, 1]
```

```
SLAU(A_c, b_c)
```

[98]:

"Нет решений"

$$\text{d) } \begin{cases} x + y + z = 1, \\ x + y + 2z = 0, \\ 2x + 2y + 3z = 0. \end{cases}$$

[99]:

```
A_d = [1 1 1; 1 1 2; 2 2 3]
```

```
b_d = [1, 0, 0]
```

```
SLAU(A_d, b_d)
```

[99]:

"Нет решений"

№3 Операции с матрицами

1. Приведите приведённые ниже матрицы к диагональному виду

a) $\begin{pmatrix} 1 & -2 \\ -2 & 1 \end{pmatrix}$

b) $\begin{pmatrix} 1 & -2 \\ -2 & 3 \end{pmatrix}$

c) $\begin{pmatrix} 1 & -2 & 0 \\ -2 & 1 & 2 \\ 0 & 2 & 0 \end{pmatrix}$

[101]:

```
function diag_matr(X)
    eigs = eigen(X)
    m_diag = diagm(eigs.values)
    return m_diag
end
```

[101]:

diag_matr (generic function with 1 method)

Рис. 32: Задание 3.1

№3 Операции с матрицами

[105]:

```
a = [1 -2; -2 1]
b = [1 -2; -2 3]
c = [1 -2 0; -2 1 2; 0 2 0];
```

[106]:

```
diag_matr(a)
```

[106]:

```
2x2 Matrix{Float64}:
-1.0  0.0
 0.0  3.0
```

[107]:

```
diag_matr(b)
```

[107]:

```
2x2 Matrix{Float64}:
-0.236068  0.0
 0.0       4.23607
```

[108]:

```
diag_matr(c)
```

[108]:

```
3x3 Matrix{Float64}:
-2.14134  0.0  0.0
 0.0      0.515138  0.0
 0.0      0.0  3.6262
```

2. Вычислите

a) $\begin{pmatrix} 1 & -2 \\ -2 & 1 \end{pmatrix}^{10}$

b) $\sqrt{\begin{pmatrix} 5 & -2 \\ -2 & 5 \end{pmatrix}}$

c) $\sqrt[3]{\begin{pmatrix} 1 & -2 \\ -2 & 1 \end{pmatrix}}$

d) $\sqrt{\begin{pmatrix} 1 & 2 \\ 2 & 3 \end{pmatrix}}$

[109]:

a = [1 -2; -2 1]

b = [5 -2; -2 5]

c = [1 2; 2 3];

Рис. 34: Задание 3.2

№3 Операции с матрицами

[110]:

```
res_a = a^10
```

[110]:

```
2x2 Matrix{Int64}:  
 29525  -29524  
-29524  29525
```

[111]:

```
res_b = sqrt(b)
```

[111]:

```
2x2 Matrix{Float64}:  
 2.1889  -0.45685  
-0.45685  2.1889
```

[112]:

```
res_c = cbrt(a)
```

[112]:

```
2x2 Matrix{Float64}:  
 0.221125  -1.22112  
-1.22112   0.221125
```

[113]:

```
res_d = sqrt(c)
```

[113]:

```
2x2 Matrix{ComplexF64}:  
 0.568864+0.351578im  0.920442-0.217287im  
 0.920442-0.217287im  1.48931+0.134291im
```

№3 Операции с матрицами

$$A = \begin{pmatrix} 140 & 97 & 74 & 168 & 131 \\ 97 & 106 & 89 & 131 & 36 \\ 74 & 89 & 152 & 144 & 71 \\ 168 & 131 & 144 & 54 & 142 \\ 131 & 36 & 71 & 142 & 36 \end{pmatrix}.$$

Создайте диагональную матрицу из собственных значений матрицы A . Создайте нижнедиагональную матрицу из матрица A . Оцените эффективность выполняемых операций.

[114]:

```
A = [140 97 74 168 131;
     97 106 89 131 36;
     74 89 152 144 71;
     168 131 144 54 142;
     131 36 71 142 36]
```

[114]:

```
5x5 Matrix{Int64}:
140  97  74 168 131
 97 106  89 131  36
 74  89 152 144  71
168 131 144  54 142
131  36  71 142  36
```

[115]:

```
eigs = eigen(A)
A_diag = diagm(eigs.values)
```

[115]:

```
5x5 Matrix{Float64}:
-128.493  0.0  0.0  0.0  0.0
  0.0 -55.8878  0.0  0.0  0.0
  0.0  0.0 42.7522  0.0  0.0
  0.0  0.0  0.0 87.1611  0.0
  0.0  0.0  0.0  0.0 512.015
```

№3 Операции с матрицами

[116]:

```
LowerTriangular(A)
```

[116]:

```
5x5 LowerTriangular{Int64, Matrix{Int64}}:
```

```
140  .  .  .  .  
97  106  .  .  .  
74  89  152  .  .  
168  131  144  54  .  
131  36  71  142  36
```

[117]:

```
@btime diag(eigs.values)
```

```
57.375 ns (2 allocations: 272 bytes)
```

[117]:

```
5x5 Matrix{Float64}:
```

```
-128.493  0.0  0.0  0.0  0.0  
0.0 -55.8878 0.0  0.0  0.0  
0.0 0.0 42.7522 0.0  0.0  
0.0 0.0 0.0 87.1611 0.0  
0.0 0.0 0.0 0.0 542.468
```

[118]:

```
@btime LowerTriangular(A)
```

```
118.708 ns (1 allocation: 16 bytes)
```

[118]:

```
5x5 LowerTriangular{Int64, Matrix{Int64}}:
```

```
140  .  .  .  .  
97  106  .  .  .  
74  89  152  .  .  
168  131  144  54  .
```

№4 Линейные модели экономики

Линейная модель экономики может быть записана как СЛАУ

$$x - Ax = y,$$

где элементы матрицы A и столбца y — неотрицательные числа. По своему смыслу в экономике элементы матрицы A и столбцов x, y не могут быть отрицательными числами.

1. Матрица A называется продуктивной, если решение x системы при любой неотрицательной правой части y имеет только неотрицательные элементы x_i . Используя это определение, проверьте, являются ли матрицы продуктивными.

a) $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$

b) $\frac{1}{2} \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$

c) $\frac{1}{10} \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$

[133]:

```
function productive(A)
    y = [1, 1]
    E = Matrix{Int}(I, 2, 2)
    x = y*((E - A)^(-1))
    if all(x .> 0)
        res = "Матрица продуктивная"
    else
        res = "Матрица непродуктивная"
    end
    return x, res
end
```

[133]:

productive (generic function with 3 methods)

```
A_a = [1 2; 3 4]
```

```
A_b = 1/2*A_a
```

```
A_c = 1/10*A_a;
```

```
[134]:
```

```
productive(A_a)
```

```
[134]:
```

```
([0.0 -0.3333333333333333], "Матрица непродуктивная")
```

```
[135]:
```

```
productive(A_b)
```

```
[135]:
```

```
([-0.25 -0.75], "Матрица непродуктивная")
```

```
[136]:
```

```
productive(A_c)
```

```
[136]:
```

```
([1.875 2.2916666666666665], "Матрица продуктивная")
```

Рис. 39: Задание 4.1

2. Критерий продуктивности: матрица A является продуктивной тогда и только тогда, когда все элементы матрица

$$(E - A)^{-1}$$

являются неотрицательными числами. Используя этот критерий, проверьте, являются ли матрицы продуктивными.

a) $\begin{pmatrix} 1 & 2 \\ 3 & 1 \end{pmatrix}$

b) $\frac{1}{2} \begin{pmatrix} 1 & 2 \\ 3 & 1 \end{pmatrix}$

c) $\frac{1}{10} \begin{pmatrix} 1 & 2 \\ 3 & 1 \end{pmatrix}$

[146]:

```
function productive2(A)
    E = Matrix{Int}(I, 2, 2)
    x = (E - A)^(-1)
    if all(x .> 0)
        res = "Матрица продуктивная"
    else
        res = "Матрица непродуктивная"
    end
    return x, res
end
```

Рис. 40: Задание 4.2

[147]:

```
A_a = [1 2; 3 1]  
A_b = 1/2*A_a  
A_c = 1/10*A_a;
```

[148]:

```
productive2(A_a)
```

[148]:

```
([-0.0 -0.3333333333333333; -0.5 0.0], "Матрица непродуктивная")
```

[149]:

```
productive2(A_b)
```

[149]:

```
([-0.39999999999999997 -0.7999999999999999; -1.2 -0.39999999999999997], "Матрица непродуктивная")
```

[150]:

```
productive2(A_c)
```

[150]:

```
([1.2 0.26666666666666666; 0.4 1.2], "Матрица продуктивная")
```

Рис. 41: Задание 4.2

3. Спектральный критерий продуктивности: матрица A является продуктивной тогда и только тогда, когда все её собственные значения по модулю меньше 1. Используя этот критерий, проверьте, являются ли матрицы продуктивными.

a) $\begin{pmatrix} 1 & 2 \\ 3 & 1 \end{pmatrix}$

b) $\frac{1}{2} \begin{pmatrix} 1 & 2 \\ 3 & 1 \end{pmatrix}$

c) $\frac{1}{10} \begin{pmatrix} 1 & 2 \\ 3 & 1 \end{pmatrix}$

d) $\begin{pmatrix} 0.1 & 0.2 & 0.3 \\ 0 & 0.1 & 0.2 \\ 0 & 0.1 & 0.3 \end{pmatrix}$

[172]:

```
function productive3(A)
    aeig = eigen(A)
    if (all(abs.(aeig.values) < 1))
        res = "Матрица продуктивная"
    else
        res = "Матрица непродуктивная"
    end
    return res
end
```

[172]:

productive3 (generic function with 1 method)

№4 Линейные модели экономики

[173]:

```
A_a = [1 2; 3 1]
A_b = 1/2*A_a
A_c = 1/10*A_a
A_d = [0.1 0.2 0.3; 0 0.1 0.2; 0 0.1 0.3];
```

[174]:

```
productive3(A_a)
```

[174]:

"Матрица непродуктивная"

[175]:

```
productive3(A_b)
```

[175]:

"Матрица непродуктивная"

[176]:

```
productive3(A_c)
```

[176]:

"Матрица продуктивная"

[177]:

```
productive3(A_d)
```

[177]:

"Матрица продуктивная"

В результате выполнения данной лабораторной работы я изучила возможности специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.

1. JuliaLang [Электронный ресурс]. 2024 JuliaLang.org contributors. URL: <https://julialang.org/> (дата обращения: 11.10.2024).
2. Julia 1.11 Documentation [Электронный ресурс]. 2024 JuliaLang.org contributors. URL: <https://docs.julialang.org/en/v1/> (дата обращения: 11.10.2024).